EFY TESTED Abhay Verma

Li-Fi ON A BUDGET: Morse Code By Flashlight

Your phone's flashlight can send a message. Within just '5 hours', you can build this 'medium' complexity project for under ₹400, using a photodiode, a comparator chip, and a Python decoder to turn Morse-code light blinks into on-screen text—all without a microcontroller.

ABHAY VERMA

Ever wanted to transmit data using nothing but pure light? Li-Fi (Light Fidelity) carries data using light instead of radio waves, and this project is a small, practical form of it: a smartphone flashlight is the transmitter, a BPW34 photodiode is the receiver, and the data is plain text sent across free space. It is a low-cost, hands-on introduction to the physics of Li-Fi.

A mobile app converts letters and numbers into Morse code and 'blinks' the flashlight accordingly. Light is switched on for a defined time to represent a 'dot' or a 'dash', a scheme called on-off keying (OOK). The base interval (one dot) is about 30ms, a dash lasts three times as long, and the gaps between symbols, letters, and words are also timed, so no separate communication protocol or framing bits are needed.

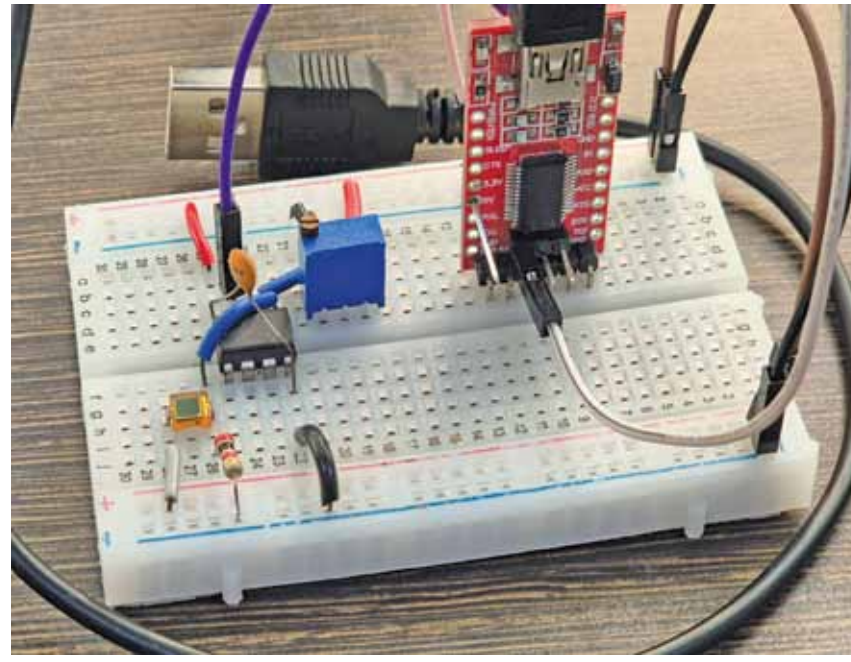


Fig. 1: Author's breadboard prototype

BILL OF MATERIALS

Component	Value/Type	Quantity	Price (₹)
Photodiode	BPW34 PIN photodiode	1	142
Dual comparator	LM393 (one section used)	1	24
USB-to-TTL adapter module	FT232RL (FTDI)	1	99
Resistor	2 kΩ, ¼ W, 5% carbon film	1	Miscellaneous 135 (enclosure, phone and computer not included)
Resistor	1 kΩ, ¼ W, 5% carbon film	1	
Variable resistor	2 kΩ	1	
Ceramic disc capacitor	0.1 μF	1	
Breadboard or general-purpose PCB	–	1	
Connecting/jumper wires	–	As needed	
Enclosure	Optional	1	
Mobile phone (transmitter)	With a Morse-code torch app	1	
Computer	Running the Python decoding program	1	
Total (excluding enclosure, phone and computer)			**345**

At the receiver, the BPW34 turns light intensity into an electrical signal, and an LM393 comparator with an adjustable threshold converts it into clean digital pulses, even under ambient lighting. The comparator output goes to the CTS pin of a USB-to-TTL adapter, so a Python program on the computer can read the pulses directly, with no microcontroller. The program measures pulse widths and gaps, rebuilds the Morse symbols, and prints the characters on the terminal in real time.

Why so slow? Transmission is